

Wordit.ai Full Working Plan

Building and evaluating a working proof of concept

September 2026 – January 2027

1. Project aim

Wordit.ai will be developed as a live biblical-reference assistant for church presentation and livestream workflows.

The proof of concept will listen to sermon audio, identify explicitly spoken Bible references, validate them against a selected Bible translation, and present the result to a human operator for approval. Once approved, the system will generate a scripture graphic and send it to a presentation or livestream system through an NDI-compatible output.

The system will assist the operator rather than replace them. Nothing should be displayed publicly without human review and approval.

2. Final product definition

By the end of January, the team should be able to demonstrate this complete workflow:

Sermon audio

- > speech recognition
- > biblical-reference detection
- > reference validation
- > pending suggestion for operator review
- > approve, edit, dismiss, or manual entry
- > scripture graphic generation
- > local preview
- > NDI output
- > presentation or livestream receiver
- > event log and evaluation results

The final product is a working proof of concept, not a production-ready commercial system. Its success will be judged by whether the workflow functions reliably, whether the operator remains in control, and whether the team can provide evidence of accuracy, latency, usability, and limitations.

3. MVP scope

Included in the MVP

- Recorded sermon audio, followed by controlled live-audio testing where practical
- Explicit biblical references such as:
 - John 3:16
 - Romans 8:28
 - Psalm 23
 - Matthew 5:3–10
 - “First Corinthians chapter thirteen, verses four to seven”
- Common book-name variations and spoken number formats
- A selected public-domain or properly licensed Bible translation
- Local Bible data for reference validation and verse retrieval
- Reference confidence and ambiguity information
- Operator review before public display
- Approve, edit, dismiss, repeat, and manual-entry controls
- Reference-only and full-verse graphics
- Private operator/cue view and public display view
- Local preview of the output
- NDI output testing with ProPresenter, OBS, or another available receiver
- Event logging for detection, approval, correction, dismissal, output, and failures
- Testing under clear, noisy, incomplete, and failure conditions

Not included in the MVP

- Understanding or interpreting the theological meaning of a sermon
- Automatically selecting related or supporting Bible passages
- Fully autonomous public publishing
- Guaranteed perfect speech recognition
- Support for every translation, language, pronunciation, or reference style
- Large-scale multi-church deployment
- Complex user accounts, cloud administration, or mobile applications unless the core workflow is already stable

4. Main workstreams

The work can be divided between team members as follows. If the team is small, responsibilities can be combined, but testing and evaluation should remain an assigned responsibility.

Workstream	Main responsibility
Workflow and research	Understand the church/livestream workflow, arrange user feedback, define test scenarios, and document limitations
Speech and language processing	Configure speech recognition, clean transcripts, recognise Bible books and spoken numbers, and produce candidate references
Bible data and validation	Select the translation, organise the local Bible data, validate references, retrieve verse text, and handle ranges
Application and backend	Connect the components, manage application state, create event records, and handle errors and fallback behaviour
Interface and user experience	Build the operator, private cue, and public display views; design clear controls and status information
Graphics and NDI	Create scripture graphics, build the local preview, connect the NDI output, and test receiver behaviour
Evaluation and documentation	Maintain the test dataset, measure accuracy and latency, run usability tests, record results, and prepare final documentation

5. Detailed monthly plan

September — Stage 1: Understand the workflow

Monthly goal

Understand how scripture is currently displayed during a service and convert the project idea into an agreed, achievable proof-of-concept specification.

Main activities

1. Speak with potential users

Where possible, speak with at least one of the following:

- Preacher

- Church presentation operator
- Livestream volunteer
- Technical director or production volunteer

Find out:

- How scripture references are currently received
- How the operator searches for and prepares a passage
- How much time is normally available
- Which mistakes are most common
- Whether the operator needs a private cue view
- Which actions must be available during a live service
- What would make the system difficult or unsafe to use

If direct access to a church user is not available, document the workflow assumptions clearly and arrange feedback from an informed reviewer before the end of the semester.

2. Document the current workflow

Create a simple current-state workflow showing:

- The preacher speaks the reference.
- The operator hears or notices the reference.
- The operator searches for the passage.
- The operator confirms the book, chapter, verse, and translation.
- The operator formats the text.
- The operator sends it to the presentation or livestream output.

Identify where delays, mistakes, and unnecessary effort occur. Use this to decide which parts Wordit.ai should assist with first.

3. Identify the technical environment

Record the available or intended:

- Microphone or mixer output
- Audio interface or capture method
- Computer hardware
- Operating system
- Presentation software
- Screens and livestream layouts
- Local network arrangement
- NDI receiver or test application
- Internet availability during a service

The team should know early whether the first prototype will be tested with recorded files, a live microphone, a mixer feed, or a simulated audio stream.

4. Define the supported reference formats

Create the first reference grammar. It should cover a limited, testable set of formats rather than every possible way of quoting scripture.

Initial formats should include:

- Book, chapter, and verse: John 3:16
- Book, chapter, and verse without punctuation: John 3 16
- Book and chapter only: Psalm 23
- Verse range: Matthew 5:3–10
- Spoken chapter and verse: “Romans chapter eight, verse twenty-eight”
- Spoken verse range: “First Corinthians chapter thirteen, verses four to seven”
- Common book aliases: “First Corinthians” and “1 Corinthians”
- Incomplete references that must be held for clarification rather than published

5. Confirm Bible data and permissions

Decide:

- Which Bible translation will be used in the MVP
- Whether the translation is public domain or properly licensed
- Whether the text can be stored locally
- Whether the text can be used in screenshots, demonstrations, and recordings
- How the translation will be identified in the interface and event log

Do not build the verse-display component around a translation whose usage rights are unclear.

6. Define the requirements and architecture

Produce a short requirements document covering:

- User roles
- Supported inputs
- Supported reference formats
- Candidate states
- Operator actions
- Display modes
- Data stored by the system
- Offline and failure behaviour
- Privacy and retention rules

- Definition of success

Create an architecture diagram showing:

- Audio input
- Speech recognition
- Reference extraction
- Bible lookup and validation
- Operator interface
- Graphics renderer
- NDI output
- Event logging

7. Prepare the evaluation foundation

Create an initial labelled dataset containing:

- Clear speech with explicit references
- Different ways of pronouncing book names
- Spoken numbers
- Chapter-only references
- Verse ranges
- Incomplete references
- Ordinary speech that resembles a Bible reference
- Examples with multiple references close together
- Noisy examples containing echo, music, or audience sound where available

Each example should record the expected result, including whether a suggestion should be created and what the correct reference is.

September deliverables

- User/workflow notes
- Current-state workflow diagram
- Technical environment inventory
- MVP requirements document
- Supported reference-format list
- Bible translation and permissions decision
- Initial architecture diagram
- Initial risk register
- Labelled test dataset
- Evaluation plan and metric definitions
- Prioritised development backlog

September targets

- At least one workflow review or informed stakeholder feedback session
- A written list of supported and unsupported reference formats
- A confirmed translation or a documented decision to use a temporary test dataset until permissions are confirmed
- At least 100 labelled test examples, split between positive and negative cases where practical
- All major technical decisions recorded with a reason for each choice

September acceptance criteria

September is complete when:

- The team can explain the user problem in practical terms.
- The first version has a clearly limited scope.
- The speech, Bible-data, interface, graphics, and NDI dependencies are identified.
- The team knows how it will measure accuracy, latency, operator correction time, and usability.
- No unresolved decision is likely to block the October detection prototype.

September weekly checkpoints

Week 1

- Confirm team members and workstream ownership.
- Arrange user or stakeholder conversations.
- Begin the workflow and technical-environment review.

Week 2

- Complete the current-state workflow.
- List user problems and required operator actions.
- Draft the supported-reference grammar.

Week 3

- Decide the translation/data approach.
- Complete the architecture diagram.
- Create the first labelled test examples.

Week 4

- Review and freeze the MVP scope.
- Finalise the evaluation plan.
- Convert the requirements into development tasks for October.

October — Stage 2: Build a reference-detection prototype

Monthly goal

Build the core pipeline that converts recorded sermon audio into validated biblical-reference suggestions.

Main activities

1. Set up the Bible data layer

- Import the selected translation into a local structured format.
- Store books, chapters, verses, and translation information.
- Support single verses, ranges, and chapter-only lookups where appropriate.
- Return readable verse text and reference labels.
- Return clear errors for invalid books, chapters, verses, and ranges.
- Add automated tests for valid and invalid references.

2. Set up speech recognition

Start with recorded audio so that tests are repeatable. The first version should be able to:

- Receive a known audio file.
- Produce a transcript.
- Preserve timestamps where available.
- Return the relevant speech segment around a detected reference.
- Record whether processing is complete, delayed, or failed.

If a live speech-recognition engine is used, document whether processing is local or external and what happens when the network is unavailable.

3. Build the reference parser

Implement deterministic parsing rules for the formats defined in September.

The parser should:

- Recognise Bible book names and aliases.

- Convert spoken numbers into numeric values.
- Identify chapters, verses, and ranges.
- Normalise different punctuation and spacing patterns.
- Distinguish a complete reference from an incomplete one.
- Preserve the original transcript text.
- Handle multiple references without silently combining them.

Use a structured candidate object containing at least:

- Original transcript segment
- Normalised book
- Chapter
- Start verse
- End verse, if present
- Translation
- Confidence score
- Detection timestamp
- Validation result
- Reason for ambiguity, if applicable

4. Add confidence and ambiguity handling

The system should not treat every detected phrase as equally reliable.

Use clear states such as:

- High-confidence candidate
- Low-confidence candidate
- Incomplete candidate
- Validated candidate
- Invalid candidate
- Duplicate candidate

A low-confidence or incomplete candidate should be shown for review or suppressed. It should not be published automatically.

5. Connect detection to validation

The detection pipeline should pass each candidate to the Bible data layer before presenting it as a usable suggestion.

Test cases should include:

- Valid book, chapter, and verse
- Valid verse range
- Invalid chapter

- Invalid verse
- Reversed verse range
- Book-name alias
- Chapter-only reference
- Incomplete reference
- Two references spoken close together

6. Create the first developer test view

Before building the final operator interface, create a simple test view or command-line result that shows:

- Transcript
- Detected candidate
- Normalised reference
- Validation result
- Confidence
- Processing time

This allows the team to identify whether errors come from speech recognition, parsing, or Bible-data validation.

7. Measure the first baseline

Run the October dataset through the prototype and record:

- Correctly detected references
- Missed references
- False suggestions
- Invalid references rejected
- Speech-recognition errors
- Parser errors
- Average and worst-case detection time

Do not combine all errors into one number. The team needs to know which part of the pipeline requires improvement.

October deliverables

- Local Bible database or structured Bible-data service
- Speech-recognition integration
- Reference parser
- Book-name and spoken-number normalisation
- Candidate event structure
- Reference validation module

- Developer test view
- Automated unit and integration tests
- Baseline accuracy and latency report
- Detection error taxonomy

October targets

- Support the reference formats agreed in September.
- Produce a structured candidate from recorded speech.
- Reject invalid references in validation tests.
- Preserve transcript context and timestamps where available.
- Establish baseline precision, recall, false-positive, false-negative, and latency measurements.
- Identify the three most common detection errors for November improvement.

October acceptance criteria

October is complete when a recorded sermon example can be processed from audio to a validated candidate reference, and the team can show the result with its confidence, transcript context, and processing time.

October weekly checkpoints

Week 1

- Set up the project environment.
- Import or connect the selected Bible data.
- Define the candidate event structure.

Week 2

- Connect speech recognition for recorded audio.
- Build book-name and number normalisation.
- Create initial parser tests.

Week 3

- Add chapter, verse, and range parsing.
- Connect the parser to Bible validation.
- Add invalid and incomplete-reference handling.

Week 4

- Run the baseline dataset.

- Categorise errors.
- Demonstrate the audio-to-candidate pipeline.
- Prepare the stable detection core for interface integration.

November — Stage 3: Create the operator interface

Monthly goal

Build a usable operator workflow that makes detected references easy to review and ensures that only approved content can reach the public display.

Main activities

1. Define the application states

The interface and backend should use clear states such as:

- Listening
- Processing
- Candidate pending
- Low confidence
- Needs clarification
- Validated
- Approved
- On air
- Dismissed
- Hidden
- Audio unavailable
- Bible data unavailable
- Output disconnected

The operator should be able to distinguish what has been detected from what has been approved and what is currently on air.

2. Build the pending-reference view

Show each candidate with:

- Reference as detected
- Normalised reference
- Relevant transcript segment

- Confidence or ambiguity indicator
- Validation result
- Translation
- Verse text preview
- Time detected
- Current status

Candidates should be ordered in a way that makes sense during a live service, normally with the newest pending item visible without hiding older unresolved items.

3. Add operator controls

Implement and test:

- Approve
- Edit
- Dismiss
- Repeat or resend
- Manual reference entry
- Hide/remove public output
- Translation selection, if more than one is available
- Reference-only or full-verse display mode

The action names should remain consistent throughout the application. For example, an operator action labelled “Approve” should produce an “Approved” state rather than a different term.

4. Separate private and public views

The private operator or preacher view may contain:

- Transcript
- Pending suggestions
- Confidence information
- Editing controls
- System-health information
- Event history

The public view should contain only the approved graphic and should not expose internal confidence scores, errors, or unfinished suggestions.

5. Add keyboard and emergency controls

Live operators may not always have time to use a mouse. Provide keyboard shortcuts where practical for:

- Selecting a pending candidate

- Approving
- Dismissing
- Hiding the output
- Opening manual entry

The emergency hide action should be visually prominent, quick to use, and available even when the normal workflow is busy.

6. Add event logging

Record the minimum information needed for testing and review:

- Candidate detected
- Candidate validated or rejected
- Candidate approved, edited, dismissed, or repeated
- Original reference and final reference
- Translation selected
- Display mode selected
- Time of each event
- Output success or failure
- Reason for an error where available

Avoid storing raw sermon audio unless permission has been obtained and the recording is needed for development or evaluation.

7. Conduct an initial usability review

Ask people who did not build the interface to complete short tasks:

- Approve a correct reference.
- Correct an incorrect verse.
- Dismiss a false suggestion.
- Enter a reference manually.
- Identify which item is currently on air.
- Hide the output during an error.
- Select the correct display mode.

Record where users hesitate, misread a status, or need explanation. Improve the interface before December integration work.

November deliverables

- Operator interface
- Pending-reference list
- Transcript or speech-segment view
- Confidence and validation indicators

- Approve, edit, dismiss, repeat, and manual-entry controls
- Private operator/cue view
- Public display placeholder or preview
- On-air indicator
- Emergency hide control
- Event log
- System-health and failure states
- Usability review findings and interface improvements

November targets

- Every detected candidate is visible as pending before approval.
- No unapproved candidate can appear in the public view.
- All required operator actions work from the interface.
- Manual entry works even when automatic detection fails.
- The operator can identify the current on-air item without assistance.
- At least two non-developers complete the core workflow with minimal guidance.

November acceptance criteria

November is complete when a non-developer can review a detected reference, correct or dismiss it, approve a valid reference, and hide the public output without needing to understand the underlying system.

November weekly checkpoints

Week 1

- Define application states and event transitions.
- Create the operator-view layout.
- Connect the detection prototype to the interface.

Week 2

- Build pending suggestions and transcript context.
- Add validation and confidence indicators.
- Implement approve, dismiss, and edit actions.

Week 3

- Add manual entry, repeat, translation, and display-mode controls.
- Separate private and public views.

- Add on-air status and emergency hide.

Week 4

- Add event logging and system-health states.
- Run usability tasks.
- Fix confusing controls and prepare the interface for graphics integration.

December — Stages 4 and 5: Generate graphics and add NDI output

Monthly goal

Create readable scripture graphics and connect approved content to a realistic presentation or livestream workflow.

Main activities

1. Design the scripture graphics

Create at least two display modes:

Reference-only display

Should show:

- Bible book
- Chapter and verse or range
- Translation identifier where required

This mode should be compact enough for a lower third or simple reference overlay.

Full-verse display

Should show:

- Verse text
- Bible reference
- Translation identifier
- Clear visual hierarchy

The design should work for both a presentation screen and a livestream layout. Avoid small text, excessive information, and layouts that are difficult to read quickly.

2. Test text wrapping and layout limits

Test:

- Short single verses
- Long verses
- Multiple verses
- Verse ranges
- Long book names
- Translation labels
- Different screen sizes
- Livestream-safe margins
- Different aspect ratios where relevant

The renderer should not allow text to be clipped or overflow outside the display area.

If a passage is too long, the system should use a defined fallback such as smaller text, multiple pages, or a clear limitation message.

3. Add local preview

The operator should be able to see the exact graphic before approving or sending it to the public output.

Clearly distinguish:

- Draft or preview
- Approved content
- On-air content
- Hidden output
- Output unavailable

4. Connect NDI output

Implement an NDI-compatible output feed or integration layer appropriate to the selected test environment.

Test:

- Receiver discovery
- Output connection
- Graphic update after approval
- Reference changes
- Full-verse changes
- Hide/remove action
- Receiver disconnection
- Receiver reconnection
- Output delay

Where a full NDI implementation is not possible, create a documented integration boundary and demonstrate the closest reliable local output available. Keep a fallback demonstration path so that an NDI issue does not prevent evaluation of the detection and operator workflow.

5. Add output and network failure behaviour

The system should:

- Show when the receiver is unavailable.
- Avoid claiming that a graphic is on air when delivery has failed.
- Preserve the approved reference in the operator view.
- Allow the operator to retry or use manual fallback.
- Record the output failure in the event log.
- Recover predictably after reconnection.

6. Run end-to-end integration tests

Use scripted scenarios including:

- One clear reference
- Two references close together
- Chapter-only reference
- Verse range
- Long verse text
- Manual entry
- Incorrect candidate corrected before approval
- Candidate dismissed
- Emergency hide
- Missing audio
- Delayed transcription
- Missing Bible data
- NDI receiver disconnected

December deliverables

- Reference-only graphic template
- Full-verse graphic template
- Text wrapping and layout handling
- Translation and reference labels
- Local preview
- Public display view
- NDI output or documented integration bridge

- Connection and disconnection handling
- Setup and deployment guide
- End-to-end integration test report

December targets

- Approved references render correctly in both display modes.
- The visual test set contains no avoidable clipping, overflow, or unreadable text.
- The local preview accurately represents the intended public output.
- The NDI receiver displays approved content in the intended test environment.
- Hide, update, and reconnect behaviour is tested.
- At least 20 scripted integration scenarios are completed and recorded.

December acceptance criteria

December is complete when an approved reference can be rendered, previewed, sent to the selected receiver, and hidden or replaced while the operator can clearly see the current output state.

December weekly checkpoints

Week 1

- Finalise graphic requirements.
- Build the reference-only and full-verse layouts.
- Test text wrapping and screen dimensions.

Week 2

- Connect graphics to approved operator actions.
- Add local preview and public display state.
- Test longer references and passages.

Week 3

- Connect the NDI output or integration bridge.
- Test receiver discovery, output, update, and hide behaviour.
- Record latency and connection issues.

Week 4

- Run scripted integration scenarios.
- Fix the highest-impact graphics and output defects.

- Complete the deployment and fallback instructions.

January — Stages 6 and 7: Test, refine, and demonstrate

Monthly goal

Evaluate the system under realistic conditions, improve the most important weaknesses, document its limitations, and prepare a repeatable final demonstration.

Main activities

1. Freeze the feature scope

At the beginning of January, stop adding major features. Only accept changes that improve:

- Detection accuracy
- Reference validation
- Operator safety
- Output reliability
- Latency
- Accessibility and readability
- Demonstration repeatability
- Documentation

New features that do not directly support the MVP should be recorded in a post-semester backlog.

2. Test live and realistic scenarios

Test slow and fast speech, different pronunciations, and different operating conditions.

The final test set should include:

- Clear speech
- Slow speech
- Fast speech
- Different pronunciations of Bible books
- Spoken numbers
- Chapter-only references
- Single verses

- Verse ranges
- Multiple references close together
- Incomplete references
- Ordinary speech that resembles a reference
- Background music
- Room echo
- Audience noise
- Delayed transcription
- Missing audio input
- Bible-data lookup failure
- NDI disconnection
- Manual-entry fallback

3. Measure the final results

Record the following metrics.

Detection accuracy

- True positives: valid spoken references detected correctly
- False positives: references suggested when no valid reference was spoken
- False negatives: valid spoken references missed by the system
- Precision: the proportion of suggestions that were correct
- Recall: the proportion of spoken references that were detected

The formulas are:

Validation accuracy

Measure whether the system correctly accepts valid references and rejects impossible book, chapter, verse, and range combinations.

Latency

Measure at least two timings:

- Detection latency: from the end of the spoken reference to the appearance of the operator suggestion
- Output latency: from operator approval to the graphic appearing on the receiver

Operator performance

Measure:

- Time to approve a correct suggestion

- Time to correct an incorrect suggestion
- Time to dismiss a false positive
- Time to use manual entry
- Time to hide the public output
- Number of usability errors during scripted tasks

Reliability

Record:

- Completed scenarios
- Failed scenarios
- Failure reason
- Whether the system recovered
- Whether the operator had a usable fallback
- Whether the public output remained safe

4. Refine the parser and interface

Prioritise improvements in this order:

- Preventing an incorrect reference from reaching the public output
- Fixing controls that prevent correction or emergency hiding
- Resolving invalid-reference and validation errors
- Improving the most common speech and parsing failures
- Reducing latency where it affects live operation
- Improving layout, readability, and convenience

5. Add offline or cached support where practical

Where feasible:

- Keep Bible data available locally.
- Cache data required for the demonstration.
- Avoid depending on the internet for basic verse lookup.
- Show a clear state when speech recognition or NDI is unavailable.
- Provide manual reference entry as a reliable fallback.

6. Document limitations and responsible use

The final documentation should explain:

- What reference formats are supported.
- Which audio conditions were tested.
- Which speech-recognition errors occurred.

- When the system produces false positives or false negatives.
- What happens when confidence is low.
- What happens when the Bible data or network is unavailable.
- Which translation was used and what permissions apply.
- Whether sermon audio is processed locally or externally.
- What data is stored and for how long.
- Why the system requires human approval.
- Why the system does not interpret sermons or act as a theological authority.

7. Prepare the final demonstration

Create a repeatable demonstration lasting approximately 8–12 minutes:

- Start the application and show system health.
- Play or speak a short sermon segment containing a clear Bible reference.
- Show the reference appearing as a pending suggestion in the private operator view.
- Display the transcript segment, validation result, and confidence information.
- Edit or confirm the reference if necessary.
- Approve the reference.
- Show the scripture graphic in the local preview and public view.
- Show the approved output on the NDI receiver.
- Demonstrate a verse range or spoken reference format.
- Demonstrate an incomplete reference or false positive being held or dismissed.
- Demonstrate manual entry or emergency hide.
- Present the final accuracy, latency, usability, and reliability results.
- State the main limitations and possible future improvements.

The demonstration should be tested at least three times from a clean start. Every step should have a fallback so that one live failure does not prevent the team from explaining the full workflow.

January deliverables

- Final tested MVP
- Final accuracy and latency report
- Final usability and reliability results
- Updated parser and interface
- Offline or cached Bible-data support where practical
- Technical architecture documentation
- Operator quick-start guide
- Setup and deployment guide

- Privacy and translation-licensing statement
- Limitations and responsible-use statement
- Final demonstration script and recording if required
- Tagged final source version
- Post-semester improvement backlog

January targets

- Complete the agreed final test matrix.
- Have no unresolved critical defect that could display unapproved or incorrect content without operator control.
- Demonstrate the complete workflow successfully at least three times.
- Report precision, recall, false positives, false negatives, invalid-reference rejection, latency, operator correction time, and failure recovery.
- Ensure every major limitation discovered during testing is documented.
- Ensure the final presentation distinguishes detected, validated, approved, and on-air content.

January acceptance criteria

January is complete when the team can run a repeatable end-to-end demonstration, provide measured results, explain known limitations, and show that the operator remains in control of every public scripture display.

6. Evaluation plan

Test dataset structure

Maintain a test dataset with the following categories:

Category	Examples
Clear positive references	Clearly spoken valid book, chapter, verse, and range references
Spoken variations	“First Corinthians,” spoken chapter numbers, spoken verse ranges
Chapter-only	References such as “Psalm 23”
Incomplete references	“Turn to Romans...” or “chapter eight...” without enough information
Invalid references	Impossible chapters, verses, or ranges
False-positive speech	Ordinary speech that resembles a reference
Multiple references	Two or more references spoken close together
Noisy audio	Echo, music, audience noise, or poor

	microphone conditions
System failures	Missing audio, missing Bible data, delayed recognition, receiver disconnection

Keep the expected answer for every example so that results can be compared consistently between October and January.

Test procedure

For each test case:

- Record the input type and test conditions.
- Run the audio through the system.
- Record the transcript and detected candidate.
- Compare the candidate with the expected reference.
- Record validation, confidence, and timestamps.
- If a candidate is produced, test the operator action required.
- Record whether the public output was correct and safe.
- Record any failure and whether the system recovered.

Suggested evaluation targets

These are initial targets and should be reviewed after the September baseline:

- Controlled-audio precision: approximately 90% or higher
- Controlled-audio recall: approximately 90% or higher
- Noisy/live-style recall: approximately 80% or higher, with limitations documented
- Invalid-reference rejection: 100% for the defined validation test set
- No automatic public display of unapproved content
- Median detection latency: approximately 5 seconds or less after the reference is spoken
- Normal local output latency: approximately 2 seconds or less after approval
- Successful completion of at least 90% of scripted end-to-end scenarios, excluding intentionally injected failures

These numbers are project targets rather than claims about the final system. The team should report the actual results honestly, including where performance falls below target.

7. Risk management

Risk	Effect	Mitigation
Speech recognition misunderstands a book name or number	Wrong or missed suggestion	Use a limited grammar, retain transcript context, show confidence, and require operator approval
Invalid reference reaches the display	Incorrect scripture shown publicly	Validate every reference against structured Bible data before approval
Operator cannot tell what is on air	Accidental or outdated display	Use separate pending, approved, and on-air states with a persistent on-air indicator
Bible translation permissions are unclear	Text cannot be used in the demonstration	Confirm the translation and permissions in September; use approved data only
Live audio is too noisy	Low accuracy during realistic testing	Start with recorded audio, build a noisy test set, and document audio requirements
Multiple references are spoken close together	Candidates are merged or missed	Use timestamps, separate candidate events, and allow operator selection
NDI integration takes longer than expected	Full output workflow is delayed	Build the local preview early, test the receiver in December, and retain a documented fallback
Network or external service fails	Detection or output stops	Prefer local/cached Bible data, show system health, and provide manual entry and local preview
Stored sermon audio creates privacy concerns	Unauthorised use or exposure of recordings	Obtain permission, minimise storage, delete temporary files, and prefer local processing where practical
Scope expands beyond the semester	Core MVP remains unfinished	Keep theology, topic inference, broad translation support, and advanced deployment in the backlog
Testing is left until January	Major defects are discovered too late	Run a baseline in October, usability testing in November, and integration tests in December

8. Quality gates

The team should not move to the next major stage until the following conditions are met.

Gate 1: September to October

- MVP scope is agreed.
- Translation/data decision is made.
- Test dataset and metrics are defined.
- Technical environment is understood.
- Detection requirements are clear.

Gate 2: October to November

- Recorded audio produces candidate references.
- Candidates are normalised and validated.
- Invalid references are rejected.
- Detection results include transcript context and confidence.

Gate 3: November to December

- Operator can approve, edit, dismiss, repeat, and manually enter references.
- Public output is separated from pending suggestions.
- Emergency hide works.
- Events are logged.

Gate 4: December to January

- Graphics render correctly.
- Local preview works.
- NDI output works in the selected test setup or the integration limitation is documented.
- Output failure and fallback states are visible.

Gate 5: Final submission

- Final test results are recorded.
- Critical safety defects are closed.
- The demonstration is repeatable.
- Documentation and limitations are complete.

9. Definition of done

A feature is complete only when it has:

- Implemented functionality

- A successful test case
- A failure or edge-case test
- A visible status or result
- An identified owner
- Documentation sufficient for another team member to use it

The project itself is complete only when the final workflow has been tested from audio input to approved output and the team can explain both its strengths and its limitations.

10. Weekly project routine

Use the following routine throughout the semester:

- Begin each week by agreeing on the specific deliverables and acceptance criteria.
- Keep tasks small enough to demonstrate during the weekly review.
- Run tests whenever a feature changes rather than waiting until the end of the month.
- Record technical decisions and reasons in the project documentation.
- Review risks every two weeks.
- Demonstrate the current working state at the end of each month.
- Keep the backlog ordered by safety, core functionality, reliability, and then polish.

At the end of each month, hold a gate review covering:

- What was completed
- What was not completed
- Evidence that the completed work functions
- Problems discovered
- Changes required for the next month
- Whether the MVP scope needs protection or adjustment

11. Priorities if the schedule becomes tight

Essential

- Explicit-reference detection
- Bible-data validation
- Operator approval workflow
- Manual entry and emergency hide
- Readable scripture output
- Evaluation results

- Human-control and safe-failure behaviour

Important

- Multiple reference formats
- Private cue view
- Full-verse display mode
- NDI integration
- Offline or cached Bible data
- Usability testing

Stretch goals

- Additional translations
- Wider pronunciation coverage
- Advanced graphics customisation
- More complex live-audio processing
- Cloud administration
- Topic-based or inferred passage suggestions

If time is lost, reduce the number of supported formats or graphic variations before removing operator approval, validation, testing, or fallback controls.

12. Final project outcome

The final Wordit.ai proof of concept should show more than an isolated speech-recognition demo or a scripture graphic.

It should demonstrate:

- ****A real user problem:**** the system supports the operator during a live scripture-display workflow.
- ****A complete technical path:**** audio is processed, a reference is detected, validated, reviewed, rendered, and output.
- ****Human control:**** the operator decides what is approved and shown publicly.
- ****Safe failure:**** uncertain references, missing data, noisy audio, and network problems do not silently produce incorrect public content.
- ****Measured performance:**** the team can show what worked, what failed, how quickly it worked, and what should be improved next.